IE 206: Introduction to AI & ML

Multiclass Classification

Manjesh K. Hanawal

February 3, 2026

Introduction

- **Multiclass Learning**: Classify an instance x into one of k possible classes.
- **Examples**:
  - Classifying documents by topic (each topic = a distinct class).
  - Determining which object appears in an image (each category of objects = a class).
- **Straightforward Approaches**:
  - Reductions to binary classification (One-Versus-All, All-Pairs).
  - Direct methods (e.g., linear multiclass predictors, multiclass SVM).

One Versus All

- Train k binary classifiers, each distinguishing "class i" vs. "not class i."
- **Training**:
  1. For each class i:
     - Label examples of class i as $+1$, all others as -1.
     - Train a binary classifier h_i.
  2. Final prediction: $\arg\max_i h_i(x)$ (or use confidence scores).
- **Pros**:
  - Simple to implement; reuse any binary classifier.
- **Cons**:
  - Each binary classifier is unaware it will be combined; can lead to suboptimal multiclass performance.

One Versus All Algorithm

One-versus-All

input:
training set $S = (x_1, y_1), \ldots, (x_m, y_m)$
algorithm for binary classification A

foreach $i \in \mathcal{Y}$:

- let $S_i = \left(x_1, (-1)^{\mathbf{1}[y_1 \neq i]}\right), \ldots, \left(x_m, (-1)^{\mathbf{1}[y_m \neq i]}\right)$
- let $h_i = A(S_i)$

output:
the multiclass hypothesis defined by

$$h(x) \in \arg\max_{i \in \mathcal{Y}} h_i(x).$$

All-Pairs

- Train $\binom{k}{2}$ binary classifiers, each discriminating between classes i and j.
- **Training**:
  1. For each pair (i, j):
     - Label examples of class i as $+1$ and class j as -1.
     - Train a binary classifier $h_{i,j}$.
  2. Final prediction: class with the most "wins" among all pairwise classifiers.
- **Pros**:
  - Often more accurate than One-versus-All on some tasks.
- **Cons**:
  - $\binom{k}{2}$ classifiers can be expensive if k is large.
  - Combining outputs can still be suboptimal.

All Pairs Algorithm

All-Pairs

input:
training set $S = (x_1, y_1), \ldots, (x_m, y_m)$
algorithm for binary classification A

foreach $i, j \in \mathcal{Y}$ such that $i < j$:

- initialize $S_{i,j}$ to be the empty sequence
- **for** $t = 1, \ldots, m$:
  - If $y_t = i$ add $(x_t, +1)$ to $S_{i,j}$
  - If $y_t = j$ add $(x_t, -1)$ to $S_{i,j}$
- let $h_{i,j} = A(S_{i,j})$

output:
the multiclass hypothesis defined by

$$h(x) \in \arg\max_{i \in \mathcal{Y}} \Big(\sum_{j \in \mathcal{Y}, j \neq i} h_{i,j}(x) \Big).$$

Problem with Reduction Approaches

Example (Multiclass categorization in $\mathbb{R}^2$)

Consider a multiclass categorization problem in which the instance space is $\mathcal{X} = \mathbb{R}^2$ and the label set is $\mathcal{Y} = \{1, 2, 3\}$. Suppose that instances of the different classes are located in nonintersecting balls, as depicted in Figure 1 below.

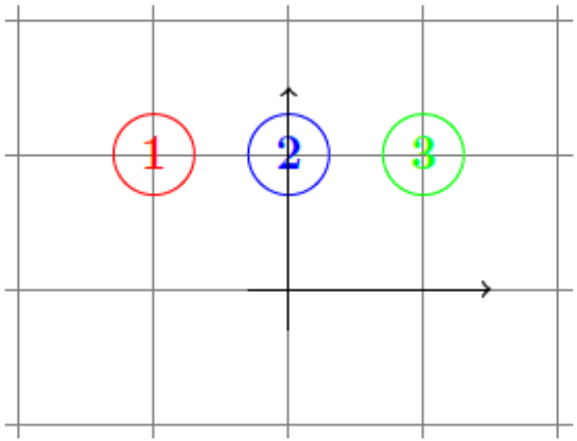

Figure: Three classes in $\mathbb{R}^2$, each in a separate nonintersecting ball.

Suppose that the probability masses of classes 1, 2, and 3 are 40%, 20%, and 40%, respectively.

Problem with One-Versus-All and All-pairs

Consider the application of One-versus-All to this problem and assume that the binary classification algorithm for distinguishing between class 2 and the rest chooses the all-negative solution (never predicting class 2). Then the final predictor might misclassify every example of class 2,

- One-Versus-All might produce trivial solutions for a low-probability class.
- A *joint* linear multiclass approach can perfectly separate all classes.

$$W_1 = \left(-\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}\right), W_2 = (0, 1), W_3 = \left(\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}\right)$$

Conclusion: Even if a perfect linear separator exists, the reduction methods might fail to find it.

Linear Multiclass Predictors (1/3)

- Instead of separate binary classifiers, learn a single-parameter vector (or set of vectors) for all classes.
- **Form**:

$$h(x) \;=\; \arg\max_{y \in \{1,\ldots,k\}} \langle w,\ \Psi(x,y) \rangle.$$

- Ψ is a class-sensitive feature mapping.
- **Example: Multi vector Construction**:
  - $\Psi(x,y)$ can be the concatenation of k copies of x, where only the y-th block is nonzero.
  - Then $\langle w, \Psi(x,y) \rangle = \langle w_y, x \rangle$, scoring each class with a dedicated sub-vector w_y.

Linear Multiclass Predictors (2/3)

A geometric illustration of the multiclass prediction over $\mathcal{X} = \mathbb{R}^2$ is given in the following figure:

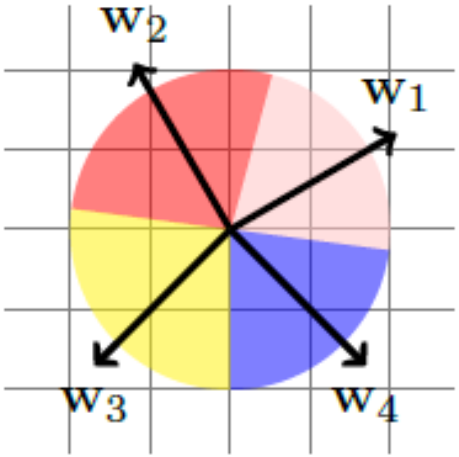

Figure: Multiclass prediction in $\mathbb{R}^2$. Each region (colored) is classified by choosing the vector $\mathbf{w}_i$ that produces the largest inner product $\langle \mathbf{w}_i, x \rangle$.

Linear Multiclass Predictors (3/3)

- **Advantages**:
  - Single model capturing relationships across classes.
  - Avoid mismatches from separately learned binary classifiers.
- **Cost-Sensitive**:
  - A general loss $\Delta(y', y)$ penalizes mistakes differently based on which pair of classes is confused.
- **ERM or Surrogate Approaches**:
  - If realizable, it can be solved with linear programs or generalized Perceptron.
  - Non-realizable: use multiclass hinge loss + convex optimization.

Multiclass SVM and SGD (1/2)

- **Multiclass SVM**:

$$\min_{w} \quad \lambda \|w\|^2 + \frac{1}{m} \sum_{i=1}^{m} \max_{y' \in \{1,\ldots,k\}} \left[\Delta(y', y_i) + \langle w, \Psi(x_i, y') - \Psi(x_i, y_i) \rangle \right].$$

- Δ is a cost function (e.g., 0–1 or cost-sensitive).
- Solved via standard convex optimization or specialized methods.

Multiclass SVM and SGD (2/2)

- **SGD approach**:
  - Uses subgradients of the hinge loss.
  - Iterative update rule:

$$w_{t+1} = w_t - \eta_t \left(\Psi(x, y^*) - \Psi(x, y)\right),$$

    where y^* maximizes the hinge expression.
  - η_t is the learning rate.
- **Benefits**:
  - Scales to large datasets.
  - Theoretically grounded with generalization bounds.

Introduction to Ranking (1/2)

- Instead of a single label, we want an *ordering* of items by "relevance."
- **Examples**:
  - Search-engine results (rank documents by relevance).
  - Fraud detection (rank transactions by suspiciousness).
- **Loss Functions**:
  - Kendall-Tau (counts pairwise disagreements).
  - NDCG (high focus on top-ranked elements).

Introduction to Ranking (2/2)

- **Linear Ranking**:

$$h_w(x_1, \ldots, x_r) = (\langle w, x_1 \rangle, \ldots, \langle w, x_r \rangle),$$

  then sort by these scores.
- **Learning**:
  - Use surrogates for ranking losses (e.g., hinge-based) + SGD.
  - Efficient algorithms rely on structure (e.g., dynamic programming, Hungarian method) to handle permutations.